a.el: Emacs Lisp Functions for Associative Data Structures

The a.el Emacs Lisp library contains functions for handling associative lists and hash tables. It provides an API in a consistent and functional way, which is greatly influenced by Clojure, dash.el and seq.el. The package is written by Arne Brasseur.

The latest tagged release of the a.el Emacs Lisp library is v0.1.1. You can use association lists, or hash tables and even vectors with certain functions. The implemented functions are pure and hence do not mutate objects. The library is released under the GNU General Public License v3.0 and it requires Emacs version 25 or later.

You can include the a.el source file to your Emacs load path and use the following Emacs Lisp code snippet to load it on to the existing running environment:

```
(load-file "/path/to/a.el")
```

Usage

We shall now explore the various functions provided by a.el using examples for both associative lists and hash tables.

You can define an associative list using the *a-list* function, and a hash table using the *a-hash-table* function. A few examples are given below to illustrate their usage:

```
(a-list key-value-pairs) ;; Syntax

(a-list :foo 1 :bar 2)
((:foo . 1) (:bar . 2))

(setq x (a-list :foo 1 :bar 2))
((:foo . 1) (:bar . 2))

(a-hash-table key-value-pairs) ;;
Syntax

(setq h (a-hash-table :a 1 :b 2))
(:a 1 :b 2)
```

The *a-associative* function will return *true* if the object passed to it is either an associative list or hash table, and *nil* otherwise. For example:

```
(a-associative? object) ;; Syntax

(a-associative? x)
t
(a-associative? h)
t

(a-associative? :foo)
nil
(a-associative? '(1 2 3))
nil
```

The Emacs Lisp source code for the *a-associative* function is provided below:

```
(defun a-associative-p (obj)
  (or (not obj)
      (hash-table-p obj)
      (and (consp obj) (consp (car
obj)))))

(defalias 'a-associative?
'a-associative-p)
```

You can retrieve a value for a specific key using the 'a-get' function as shown below:

```
(a-get map key) ;; Syntax

(a-get x :foo)
1
(a-get x :bar)
2
(a-get h :a)
1
(a-get h :notexist)
nil
```

The keys in the associative list or hash table can be obtained using the *a-keys* function, while the values can be retrieved using the *a-vals* function, as illustrated below:

```
(a-keys collection) ;; Syntax

(a-keys x)
(:foo :bar)

(a-keys h)
(:a :b)

(a-vals collection) ;; Syntax

(a-vals x)
```